23CSE202 DATABASE MANAGEMENT SYSTEMS

S3 B.Tech CSE

Department of Computer Science and Engineering

Amrita School of Computing, Amritapuri Campus

3. Database Design Document

Group No	C4
Project Title	Apex Bank Management System

Team Members

Roll No	Student Name
AM.SC.U4CSE25225	Kaki Venkata Pavithra
AM.SC.U4CSE25250	Singuru Chathurya
AM.SC.U4CSE25257	Harshini Vedulla
AM.SC.U4CSE25268	Gara Sriha

Project Overview

Apex Bank Management System is a relational-database-backed retail banking platform that connects Customers, Employees and Administrators on a single system, replacing manual, ledger-based record keeping with automated account management, fund transfers, debit-card issuance and loan processing. This document focuses on the database layer that underpins that platform — the ER model, the relational schema derived from it, and the normalization analysis that justifies the schema's design.

Contents of this Document

Section	Title	Covers
3.1	Detailed Description of the Project	Scenario, entities/attributes, users and business rules
3.2	ER Diagram	Chen-notation ER model with cardinality and participation constraints
3.3	Relational Schema	10 relations mapped from the ER diagram, with keys and constraints
3.4	Normalization & FD Analysis	Functional dependencies, candidate-key closure, and 1NF→BCNF decomposition

Reference Material

This design document should be read alongside the Project Abstract (problem statement and objectives) and the UI Design Document (functional/non-functional requirements and screen designs) submitted separately by Group C4.

GitHub repository: https://github.com/pavithra061108/DBMS-UI-DESIGN

Live demo: https://pavithra061108.github.io/DBMS-UI-DESIGN/

Database Design Highlights

- Role-based access (Customer, Employee, Admin) modelled with disjoint, total EER specialisation on a single USER entity.
- Historical ledger integrity preserved by treating TRANSACTION as an append-only weak entity — rows are never updated or deleted, and BalanceAfter snapshots the running balance at the time of each entry.
- Every relation in the final schema is verified to be in BCNF (Section 3.4), with CARD split out as a lossless, dependency-preserving refinement of ACCOUNT.

3.1 Detailed Description of the Project

The Apex Bank Management System is a database-driven application that automates core retail-banking operations — account opening, deposits and withdrawals, fund transfers, debit-card issuance, loan processing and branch administration — replacing manual, ledger-based record keeping with a single relational platform. Three categories of users interact with the system: Customers, Employees (branch staff who service accounts and process loans) and Administrators (who audit and manage the system). The database is the backbone of the system: it stores and relates data about user identities, branches, accounts, transactions, nominees, cards and loans, while enforcing integrity constraints (primary/foreign keys, uniqueness, domain and check constraints) so that financial data stays accurate, consistent and non-redundant.

Entities and their attributes.

- **User** — every person who can log in (UserID, Name, Email, Phone, Password, Role, CreatedAt). Role-specific data is captured through specialisation into Customer (Address, City, DateOfBirth, KYCStatus), Employee (BranchID, Designation, Salary) and Admin (AccessLevel).
- **Branch** — a physical bank branch (BranchID, BranchName, IFSCCode, Address, City, ContactNumber, IsActive).
- **Account** — a bank account opened by exactly one customer at exactly one branch (AccountID, AccountNumber, AccountType, Balance, OpenedDate, Status).
- **Transaction** — a single ledger entry against one account (TransactionSeqNo, AccountID, TransactionType, Amount, TransactionDate, BalanceAfter, Description).
- **Nominee** — a beneficiary registered against one account (NomineeSeqNo, AccountID, NomineeName, Relationship, SharePercentage).
- **Card** — the debit card issued for exactly one account (CardID, AccountID, CardNumber, CardType, ExpiryDate, CVV, Status).
- **Loan** — a loan applied for by one customer and processed by one employee (LoanID, CustomerID, EmployeeID, LoanType, PrincipalAmount, InterestRate, TenureMonths, LoanStatus, DisbursedDate).

Business rules.

- Every USER row belongs to exactly one of three roles (disjoint, total specialisation); role-specific attributes live only in the matching subclass table.
- An Employee is affiliated with exactly one Branch, but a Branch may employ many Employees.
- A Customer may own zero or more Accounts; each Account is owned by exactly one Customer and is opened at exactly one Branch (joint accounts are out of scope for this design).

- An Account accumulates zero or more Transactions; a Transaction cannot exist without referencing a valid Account. Once written, a Transaction row is never updated or deleted — the ledger is append-only, and the current Balance on Account is a maintained summary of its Transactions.
- An Account may register zero or more Nominees; a Nominee cannot exist without a valid Account, and the SharePercentage values of all Nominees on one Account must sum to 100 (enforced by application logic / a trigger).
- Every Account has exactly one Card and every Card belongs to exactly one Account (1:1, mandatory on both sides) — an account cannot be considered fully activated without a card, and a card cannot be issued without an account.
- A Customer may apply for zero or more Loans; each Loan is applied for by exactly one Customer and is processed by exactly one Employee, while one Employee may process many Loans over time.
- LoanStatus follows the sequence Applied → Under Review → Approved → Disbursed → Closed (or Rejected / Defaulted at intermediate points); this transition is kept consistent through application logic / a database trigger.
- Administrators do not take part in the transactional relationships above; they hold unrestricted read access for auditing and reporting (transaction, loan and user reports) and can freeze/deactivate any User account or Account.

3.2 ER Diagram

Chen notation is used: rectangles are entities (double-bordered = weak entity), diamonds are relationships, and ellipses are attributes (key attributes underlined, partial-key/discriminator attributes shown with a dashed outline). USER is modelled with disjoint, total specialisation into CUSTOMER, EMPLOYEE and ADMIN based on the Role attribute; ADMIN is not linked by any transactional relationship since it only performs cross-cutting auditing/management.

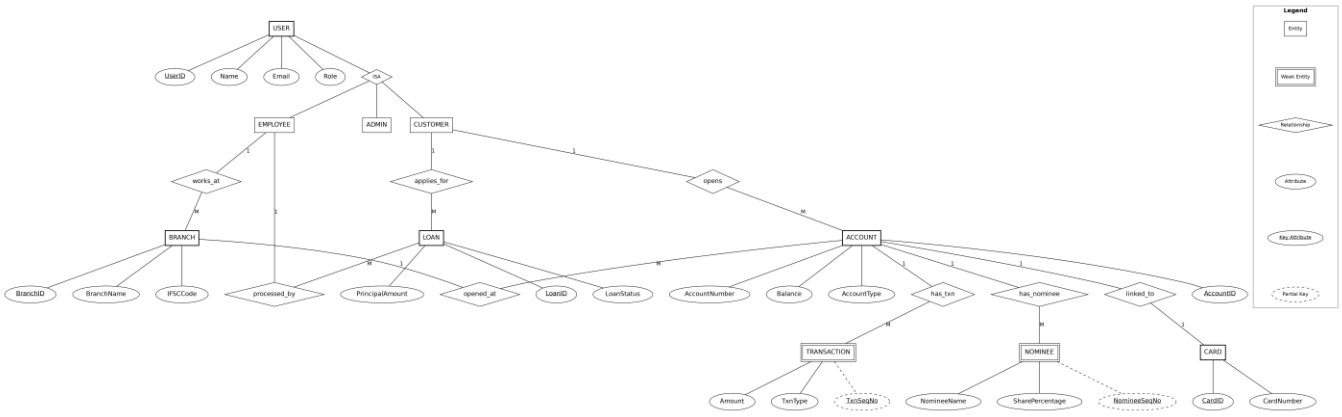


Fig. 3.1 — ER Diagram (Chen notation) of the Apex Bank Management System database

Cardinality and participation summary

Relationship	Entities	Cardinality	Participation
works_at	EMPLOYEE – BRANCH	M : 1	Total (Employee) / Partial (Branch)
opens	CUSTOMER – ACCOUNT	1 : M	Total (Account) / Partial (Customer)
opened_at	ACCOUNT – BRANCH	M : 1	Total (Account) / Partial (Branch)
has_txn	ACCOUNT – TRANSACTION	1 : M	Total (Transaction) / Partial (Account)
has_nominee	ACCOUNT – NOMINEE	1 : M	Partial (both sides)
linked_to	ACCOUNT – CARD	1 : 1	Total (both sides)
applies_for	CUSTOMER – LOAN	1 : M	Total (Loan) / Partial (Customer)
processed_by	LOAN – EMPLOYEE	M : 1	Total (Loan) / Partial (Employee)

3.3 Relational Schema

The ER diagram is mapped to relations using standard rules: each strong entity becomes a relation keyed on its identifier; the total, disjoint specialisation of USER is implemented with one relation per subclass, each sharing UserID as both primary key and foreign key back to USER; each weak entity (TRANSACTION, NOMINEE) becomes a relation whose primary key is the composite of its own discriminator and the owner entity's key; 1:M relationships place the foreign key on the 'many' side (EMPLOYEE.BranchID, ACCOUNT.CustomerID/BranchID, LOAN.CustomerID/EmployeeID); and the mandatory 1:1 relationship (Account–Card) places a unique foreign key on the dependent relation.

USER

Attribute	Type	Key	Constraint / References
UserID	INT	PK	—
Name	VARCHAR(80)	—	NOT NULL
Email	VARCHAR(100)	—	UNIQUE, NOT NULL
Phone	VARCHAR(15)	—	NOT NULL
Password	VARCHAR(255)	—	NOT NULL (hashed)
Role	ENUM	—	CHECK IN (Customer, Employee, Admin)
CreatedAt	DATETIME	—	DEFAULT current time

CUSTOMER

(subclass of USER)

Attribute	Type	Key	Constraint / References
UserID	INT	PK, FK	References USER(UserID)
Address	VARCHAR(150)	—	—
City	VARCHAR(50)	—	—
DateOfBirth	DATE	—	—

Attribute	Type	Key	Constraint / References
KYCStatus	VARCHAR(15)	—	CHECK IN (Pending, Verified, Rejected)

EMPLOYEE

(subclass of USER)

Attribute	Type	Key	Constraint / References
UserID	INT	PK, FK	References USER(UserID)
BranchID	INT	FK	NOT NULL, References BRANCH(BranchID)
Designation	VARCHAR(40)	—	—
Salary	DECIMAL(10,2)	—	CHECK > 0

ADMIN

(subclass of USER)

Attribute	Type	Key	Constraint / References
UserID	INT	PK, FK	References USER(UserID)
AccessLevel	VARCHAR(20)	—	—

BRANCH

Attribute	Type	Key	Constraint / References
BranchID	INT	PK	—
BranchName	VARCHAR(100)	—	NOT NULL
IFSCCode	VARCHAR(11)	—	UNIQUE, NOT NULL
Address	VARCHAR(150)	—	NOT NULL
City	VARCHAR(50)	—	NOT NULL
ContactNumber	VARCHAR(15)	—	—
IsActive	BOOLEAN	—	DEFAULT TRUE

ACCOUNT

Attribute	Type	Key	Constraint / References
AccountID	INT	PK	—
AccountNumber	VARCHAR(18)	—	UNIQUE, NOT NULL

Attribute	Type	Key	Constraint / References
CustomerID	INT	FK	NOT NULL, References CUSTOMER(UserID)
BranchID	INT	FK	NOT NULL, References BRANCH(BranchID)
AccountType	VARCHAR(20)	—	CHECK IN (Savings, Current, FixedDeposit)
Balance	DECIMAL(12,2)	—	DEFAULT 0, CHECK >= 0
OpenedDate	DATETIME	—	DEFAULT current time
Status	VARCHAR(15)	—	CHECK IN (Active, Frozen, Closed)

TRANSACTION

(weak entity — owner: ACCOUNT)

Attribute	Type	Key	Constraint / References
AccountID	INT	PK, FK	References ACCOUNT(AccountID)
TransactionSeqNo	INT	PK	discriminator, sequential per account
TransactionType	VARCHAR(15)	—	CHECK IN (Deposit, Withdrawal, TransferIn, TransferOut)
Amount	DECIMAL(12,2)	—	CHECK > 0
TransactionDate	DATETIME	—	DEFAULT current time
BalanceAfter	DECIMAL(12,2)	—	snapshot of balance after this entry
Description	VARCHAR(255)	—	—

NOMINEE

(weak entity — owner: ACCOUNT)

Attribute	Type	Key	Constraint / References
AccountID	INT	PK, FK	References ACCOUNT(AccountID)
NomineeSeqNo	INT	PK	discriminator
NomineeName	VARCHAR(80)	—	NOT NULL
Relationship	VARCHAR(30)	—	—
SharePercentage	DECIMAL(5,2)	—	CHECK 0–100

CARD

Attribute	Type	Key	Constraint / References
CardID	INT	PK	—
AccountID	INT	FK	UNIQUE, NOT NULL, References ACCOUNT(AccountID)

Attribute	Type	Key	Constraint / References
CardNumber	VARCHAR(16)	—	UNIQUE, NOT NULL
CardType	VARCHAR(10)	—	CHECK IN (Debit, Credit)
ExpiryDate	DATE	—	NOT NULL
CVV	VARCHAR(255)	—	NOT NULL (hashed)
Status	VARCHAR(10)	—	CHECK IN (Active, Blocked, Expired)

LOAN

Attribute	Type	Key	Constraint / References
LoanID	INT	PK	—
CustomerID	INT	FK	NOT NULL, References CUSTOMER(UserID)
EmployeeID	INT	FK	NOT NULL, References EMPLOYEE(UserID)
LoanType	VARCHAR(20)	—	CHECK IN (Personal, Home, Auto, Education)
PrincipalAmount	DECIMAL(12,2)	—	CHECK > 0
InterestRate	DECIMAL(4,2)	—	CHECK > 0
TenureMonths	INT	—	CHECK > 0
LoanStatus	VARCHAR(20)	—	CHECK IN (Applied, UnderReview, Approved, Rejected, Disbursed, Closed, Defaulted)
DisbursedDate	DATETIME	—	NULL until Disbursed

3.4 Normalization and Functional Dependency Analysis

To demonstrate the normalization process, consider the flat Transaction Statement Report relation that a naive, unnormalised design might use for generating account statements (the same relation the platform must never store, since it is exactly the redundancy the DBMS design in Section 3.3 is built to avoid):

> *STATEMENT_REPORT (TransactionID, TransactionDate, AccountID, AccountNumber, CustomerID, CustomerName, CustomerPhone, BranchID, BranchName, BranchCity, TransactionType, Amount, EmployeeID, EmployeeName)*

Functional dependencies identified

FD	Dependency
FD1	TransactionID → TransactionDate, AccountID, TransactionType, Amount, EmployeeID
FD2	AccountID → AccountNumber, CustomerID, BranchID
FD3	CustomerID → CustomerName, CustomerPhone
FD4	BranchID → BranchName, BranchCity
FD5	EmployeeID → EmployeeName

Candidate key justification using attribute closure

Every attribute of STATEMENT_REPORT is determined, directly or transitively, by TransactionID alone (a transaction is processed by one employee, against one account, which belongs to one customer at one branch — no attribute needs any other attribute in combination with TransactionID). This suggests {TransactionID} as the candidate key; the closure computation confirms it.

Step 1 — Closure of {TransactionID}:

- $\{TransactionID\}^+$ = { TransactionID }
- apply FD1 → + TransactionDate, AccountID, TransactionType, Amount, EmployeeID
- apply FD2 (AccountID present) → + AccountNumber, CustomerID, BranchID
- apply FD3 (CustomerID present) → + CustomerName, CustomerPhone
- apply FD4 (BranchID present) → + BranchName, BranchCity
- apply FD5 (EmployeeID present) → + EmployeeName

$\{TransactionID\}^+$ = ALL 14 attributes of STATEMENT_REPORT ➤ {TransactionID} is a super key. Since it is a single attribute, it is trivially minimal, so {TransactionID} is the only candidate key and therefore the primary key.

1NF — First Normal Form

STATEMENT_REPORT is organised so that each row corresponds to exactly one transaction and every attribute holds a single atomic value — there is no repeating group such as a comma-separated list of transactions inside one AccountID row. The relation therefore satisfies 1NF.

2NF — Second Normal Form

The candidate key {TransactionID} is a single attribute, so no partial dependency is possible (a partial dependency requires a composite key). Every non-key attribute already depends on the whole key, so STATEMENT_REPORT trivially satisfies 2NF. However, transitive dependencies through AccountID, CustomerID, BranchID and EmployeeID remain, which 3NF must address.

3NF — Third Normal Form

Inside STATEMENT_REPORT, TransactionID → AccountID and AccountID → AccountNumber, CustomerID, BranchID, so TransactionID determines AccountNumber/CustomerID/BranchID transitively through the non-key attribute AccountID. The same transitive pattern holds for CustomerID → CustomerName/CustomerPhone, BranchID → BranchName/BranchCity and EmployeeID → EmployeeName. These transitive dependencies violate 3NF.

Decomposition to remove transitive dependencies:

- TRANSACTION_3NF (TransactionID, TransactionDate, AccountID, TransactionType, Amount, EmployeeID) — key: TransactionID
- ACCOUNT_3NF (AccountID, AccountNumber, CustomerID, BranchID) — key: AccountID
- CUSTOMER_3NF (CustomerID, CustomerName, CustomerPhone) — key: CustomerID

- BRANCH_3NF (BranchID, BranchName, BranchCity) — key: BranchID
- EMPLOYEE_3NF (EmployeeID, EmployeeName) — key: EmployeeID

Every relation now has no transitive dependency on a non-key attribute — all relations are in 3NF.

BCNF — Boyce–Codd Normal Form

A relation is in BCNF when the determinant of every non-trivial FD is a super key. Checking each relation from the 3NF decomposition:

Relation	FD(s) present	Determinant is key?
TRANSACTION_3NF	TransactionID → all other attributes	Yes — BCNF ✓
ACCOUNT_3NF	AccountID → AccountNumber, CustomerID, BranchID	Yes — BCNF ✓
CUSTOMER_3NF	CustomerID → Name, Phone	Yes — BCNF ✓
BRANCH_3NF	BranchID → Name, City	Yes — BCNF ✓
EMPLOYEE_3NF	EmployeeID → Name	Yes — BCNF ✓

Every relation now has a single determinant that is also its key, so no further decomposition is required on FD grounds — the schema is in BCNF.

Mapping back to the final schema (Section 3.3)

This textbook decomposition is exactly what motivates the final design: TRANSACTION_3NF → TRANSACTION (re-keyed on the composite {AccountID, TransactionSeqNo} once modelled as a weak entity), ACCOUNT_3NF → ACCOUNT (extended with BranchID and AccountType/Balance/Status), CUSTOMER_3NF → CUSTOMER (extended with UserID through USER specialisation), BRANCH_3NF → BRANCH and EMPLOYEE_3NF → EMPLOYEE (extended with UserID through USER specialisation).

CARD and NOMINEE were not part of the original flat report (a statement report has no reason to carry card or nominee details), so they do not appear in the FD analysis above; they were added directly at the ER stage as entities in their own right — CARD in a mandatory 1:1 relationship with ACCOUNT, and NOMINEE as a weak entity of ACCOUNT — because neither is functionally determined by TransactionID or by any attribute of the statement report. Keeping them as separate relations, each keyed on its own identifier (or composite key, for the weak entity), is lossless and avoids repeating card or nominee data across every transaction row, which is the same normalisation principle applied one level up.